

```
PyList& PyList::operator=(const PyList &rhs) {
    if (this != &rhs) {        // a = a would free its own array
        delete [] myArray;
        mySize      = rhs.mySize;
        myCapacity = rhs.myCapacity;
        myArray     = new Item[myCapacity];
        for (int i = 0; i < mySize; ++i)
            myArray[i] = rhs.myArray[i];
    }
    return *this;                // so that a = b = c works
}
```

The rule of three. A class that needs one of the destructor, the copy constructor or the assignment operator needs all three. All three answer the same question: who frees this block?

Beat 6. Close (3 min)

PyList can now be copied, assigned and destroyed without losing an array or freeing one twice.

Monday: **PyList** stops caring whether it holds ints.

CS112: Friday 2 October, 12:15 to 1:20

Wednesday our `append` stopped copying everything. It doubles the room when it runs out, and the debugger showed the address in the watch window changing under us as the array moved.

Today: nothing ever gives that array back. And copying a **PyList** copies the address, so two objects end up owning one block. Three functions fix both.

TIME	BEAT
12:15	1. It leaks right now
12:29	2. <code>v[2]</code> , and the <code>this</code> pointer
12:41	3. The copy that bites
12:55	4. Our own block: the copy constructor
13:07	5. <code>operator=</code> and the rule of three
13:17	6. Close

No quiz today.

Beat 1. It leaks right now (14 min)

Talk to your neighbor. A **PyList** goes out of scope at the end of a function. What happens to the array it asked for?

Nothing complains. The compiler is happy and `make test` still passes, because a leak is not an error. It only shows up in how much memory the process is holding:

```
int main() {
    for (int i = 0; i < 200000; ++i) {
        PyList v;
        for (int j = 0; j < 100; ++j) v.append(j);
    }
}
```

Run that with Activity Monitor open on the Memory tab. The number climbs and never comes back down.

What one line fixes this, and where does it go?

```
PyList::~PyList() { delete [] myArray; myArray = nullptr; }
```

The compiler writes a destructor for you and it destroys the **members**. It destroys the pointer. It does not touch what the pointer points at.

If your class calls `new`, your class defines a destructor.

Same loop again. The number sits flat.

Beat 2. `v[2]`, and the `this` pointer (12 min)

`getIndex(2)` works, and nobody writes a list that way. Python says `v[2]`.

```
Item& PyList::operator[](int i) {  
    return myArray[i];  
}
```

Why `Item&` and not `Item`?

Return `Item` and the call hands back a copy, so `v[2] = 99` assigns to a temporary that is thrown away a line later. `Item&` makes `v[2]` another name for the slot itself, which is what puts it on the left of an `=`.

Inside that function, `myArray` is short for `this->myArray`. Every member function is handed a pointer to the object it was called on, silently, named `this`.

```
v[2] = 99;           // calls v.operator[](2), with this == &v
```

Python writes that pointer out in every signature and calls it `self`. C++ passes it without asking and names it `this`.

Beat 3. The copy that bites (14 min)

```
PyList a;  
a.append(3);  
a.append(7);
```

```
PyList b = a;           // what exactly got copied?
```

Talk to your neighbor. `b` is a new object with its own three members. What is in them?

On the board: two objects on the stack, one arrow out of each, both arrows landing on the same block.

```
free(): double free detected in tcache 2  
Aborted (core dumped)
```

Two faults from one line. `b[0] = 9` changes `a[0]`. The second destructor frees a block that is already gone, which is the crash above. Same crash as the lab, returning a list by value.

Beat 4. Our own block: the copy constructor (12 min)

What has to happen instead, for `b` to be independent?

```
PyList::PyList(const PyList &orig) {  
    mySize    = orig.mySize;  
    myCapacity = orig.myCapacity;  
    myArray    = new Item[myCapacity]; // our own block  
    for (int i = 0; i < mySize; ++i)  
        myArray[i] = orig.myArray[i]; // copy the values  
}
```

Same moves as Wednesday's growth code: allocate, copy across, own it. Now each object has its own block.

It runs in three places, and only the first one looks like a copy:

```
PyList b = a;           // 1. written down  
void show(PyList v);     // 2. passed by value  
PyList makeList();       // 3. returned by value
```

Which of the three have we already written without noticing?

Beat 5. `operator=` and the rule of three (10 min)

`b = a;` on an object that already exists is not the copy constructor at all. It is assignment, and the default one copies the address the same way.